

Experiment 01 · UART – Asynchronous Serial Link

Two NUCLEO-F446RE boards, one command byte over USART1 in asynchronous mode

NUCLEO-F446RE
× 2
STM32F446RET6,
Cortex-M4

USART1 (APB2)
peripheral used

9600 Bd, 8N1
line settings

2 (TX→RX, GND)
wires between boards

Bare metal
CMSIS registers, no
HAL, no CubeMX

1 Aim and specification

Two NUCLEO-F446RE boards talk to each other over **UART** and nothing else. The board holding the **master** listing reads its blue B1 user button and measures how long it is held; the board holding the **slave** listing has no button and reacts only to what arrives on the UART link, blinking its green LD2 in one of two clearly distinguishable patterns. After programming, both boards run from any 5 V USB supply with no PC attached.

1.1 Behavioural specification

Stimulus on the master	Measured press	Byte transmitted	Slave LD2 pattern	LED-to-LED spacing	Total
Short click	< 1000 ms	'S' (0x53)	3 blinks · 150 ms ON / 150 ms OFF	300 ms	0.9 s
Press and hold	≥ 1000 ms	'L' (0x4C)	3 blinks · 1000 ms ON / 1000 ms OFF	2000 ms	6.0 s

The long-press pattern is the point of the experiment: 1000 ms on plus 1000 ms off means the slave LED starts a blink every **2.0 s**, against 300 ms for a short click. Both numbers are the two literals in the slave's `blink()` call, so the timing can be retuned by editing one line. The master flashes its own LD2 for 100 ms each time it sends, so you can always see which board acted.

1.2 How short these programs are

Listing	Lines of code	What it does
Master	about 30	clock enables, four GPIO lines, peripheral init, one loop
Slave	about 25	clock enables, two GPIO lines, peripheral init, one loop

There is no interrupt handler, no clock-tree configuration, no acknowledgement protocol and no checksum. Everything that remains is something an examiner can point at and ask you to justify – which is exactly what you want when you are typing from memory.

2 Apparatus

Qty	Item	Notes
2	NUCLEO-F446RE development board	one typed with the master listing, one with the slave listing
1 set	Male-male jumper wires (Dupont)	keep them under about 20 cm
2	USB Mini-B cables	for programming, then for standalone power
1–2	USB power bank or 5 V phone charger	to run the boards away from the PC
2	4.7 kΩ resistors + small breadboard	I2C experiment only – mandatory bus pull-ups

3 Pin assignment

MCU pin	Function	Configuration	Morpho	Arduino	Used by
PA9	USART1_TX	AF7	CN10-21	D8	master → slave
PA10	USART1_RX	AF7	CN10-33	D2	slave only
PC13	B1 user button	input + pull-up	CN7-23	-	master only
PA5	LD2 user LED	output	CN10-11	D13	both boards
GND	0 V reference	-	CN10-20	-	mandatory

Reading the headers

CN7 and CN10 are the two 2×19 “morpho” headers along the long edges. Pin 1 of each is the row nearest the ST-LINK part of the board; odd numbers are in the left column and even numbers in the right column when the board is seen from above with the USB connector upwards. Every pin name is printed on the underside of the PCB – when in doubt, find a pin by its *name*, not by counting rows.

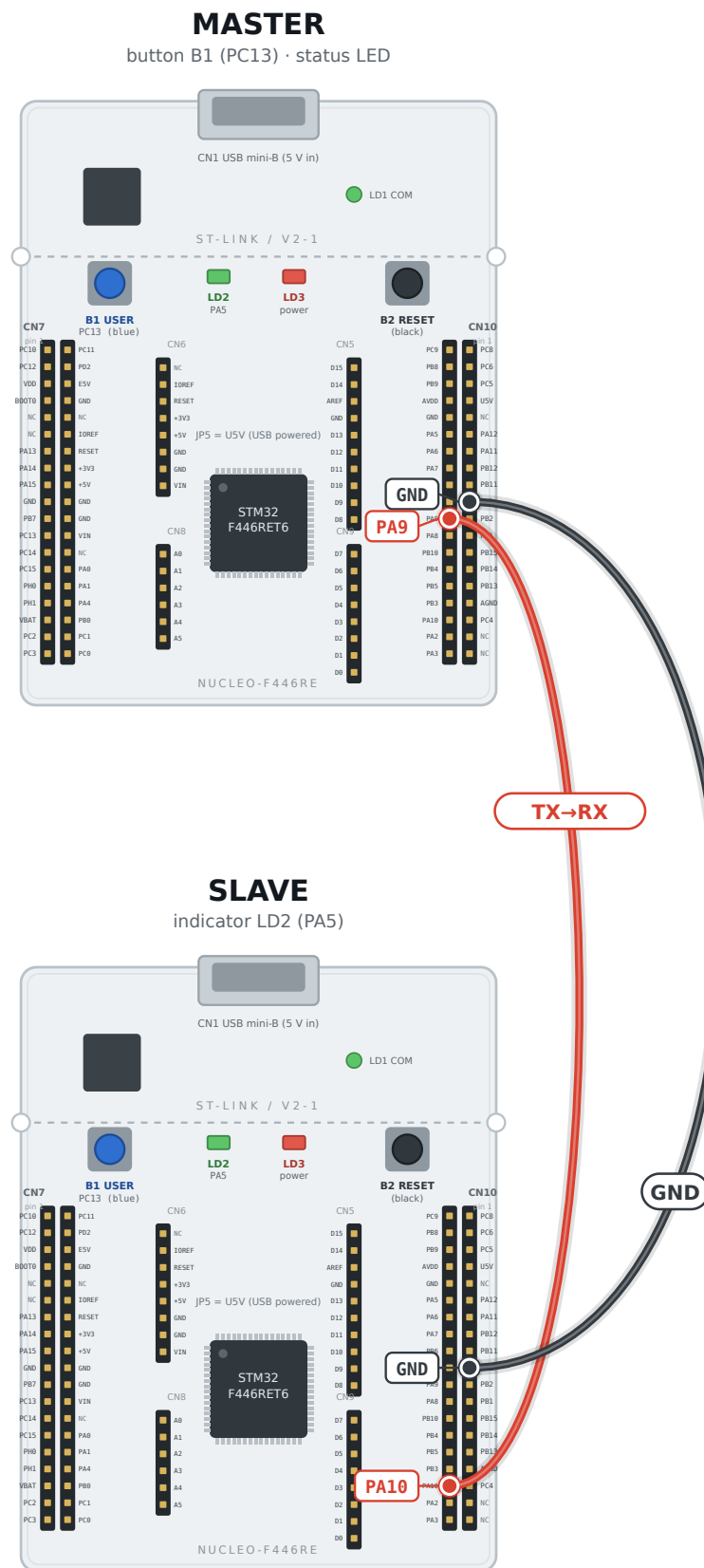
4 Circuit and wiring

Make every connection with the boards **unpowered**. The colours in the table are the ones used in Figure 1 on the next page; any colour works electrically, but keeping a convention makes a wiring fault obvious at a glance.

Signal	MASTER pin	SLAVE pin	Wire colour used	Topology
Command byte	PA9 · CN10-21	PA10 · CN10-33	red	crossed
Ground	GND · CN10-20	GND · CN10-20	black	common

Experiment 1 · UART (USART1 asynchronous) - board-to-board wiring

Top view, USB connectors upwards. Pin names are silk-screened on the underside of each board.



Only two wires. TX goes to RX - the link is CROSSED - and the grounds must be joined or the receiver has no TX. PA9 = CN10-21 = Arduino D8 | PA10 = CN10-33 = Arduino D2 | GND = CN10-20.

Figure 1 - Physical wiring between the two boards, drawn to the real board outline. The coloured rings mark the pads that carry a wire.

5 Theory of operation

An asynchronous UART link carries **no clock**. The two ends agree on a bit period in advance; the receiver re-synchronises on the falling edge of every start bit and samples each bit at its centre with a clock 16 times faster than the bit rate. This is why the baud rate must be identical on both boards – an error beyond roughly $\pm 2\%$ corrupts the last bits of every character.

The bit period comes from one register. `BRR` holds f_{PCLK2} / baud in a 12.4 fixed-point format, which for this experiment is

$$\text{BRR} = 16\,000\,000 / 9600 = 1666.67 \approx 1667$$

giving $16\,000\,000 / 1667 = 9598 \text{ Bd}$, an error of 0.02 %. Everything else in the peripheral is left at its reset value: 8 data bits, no parity, one stop bit – the familiar 8N1 – so `CR2` and `CR3` are never written at all.

Why the command is a single ASCII byte

The master sends 'S' or 'L' and nothing else. One byte is enough because the slave has only two possible reactions, and using printable characters means you can drop a USB-serial adapter on the wire and read the traffic directly. The slave treats anything that is not 'L' as a short command, so a corrupted byte produces the harmless pattern rather than a lock-up.

Why there is no clock-configuration code

After reset the STM32F446 runs from its internal 16 MHz RC oscillator (HSI) with every bus prescaler at /1, and CubeIDE's `SystemInit()` does not change that – it only enables the FPU. So **SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz** without a single line of setup. That single fact is where the two constants in the listing come from: 16000 in `delay_ms()` (16 MHz / 1 kHz) and the peripheral timing value in the init. Do not paste a `SystemClock_Config()` from a HAL example into these projects. If you raise SYSCLK, the delays and the baud rate move with it.

5.2 The delay function, without an interrupt

All eight listings share the same six-line timer. SysTick is a 24-bit down-counter inside the Cortex-M4 core, so it needs no RCC clock enable and no NVIC setup:

```
SysTick->LOAD = 16000 - 1; /* reload = 1 ms at 16 MHz */
SysTick->VAL = 0;          /* clear the counter */
SysTick->CTRL = 5;         /* bit0 ENABLE, bit2 CLKSOURCE = core clock */
while (ms-- > 0) while (!(SysTick->CTRL & (1 << 16))); /* COUNTFLAG */
SysTick->CTRL = 0;
```

COUNTFLAG (bit 16) is set every time the counter reaches zero and is **cleared by reading CTRL**, so the inner loop consumes exactly one millisecond per pass. Because there is no interrupt there is no `SysTick_Handler`, no `volatile` global and no chance of forgetting the handler name – three fewer things to get wrong under exam pressure.

The press length is then measured by counting those milliseconds while the button is held: `while (!(GPIOC->IDR & (1 << 13))) { delay_ms(10); t += 10; }`, giving a resolution of 10 ms, which is far finer than the 1000 ms decision threshold.

5.4 Register configuration summary

Register	Value written	Why
<code>RCC->AHB1ENR</code>	<code>(1 << 0) (1 << 2)</code>	GPIOA and GPIOC clocks – a port with no clock ignores every write
<code>RCC->APB2ENR</code>	<code>(1 << 4)</code>	USART1 clock
<code>GPIOC->PUPDR</code>	<code>(1 << 26)</code>	PC13 pull-up, so the released button reads as 1

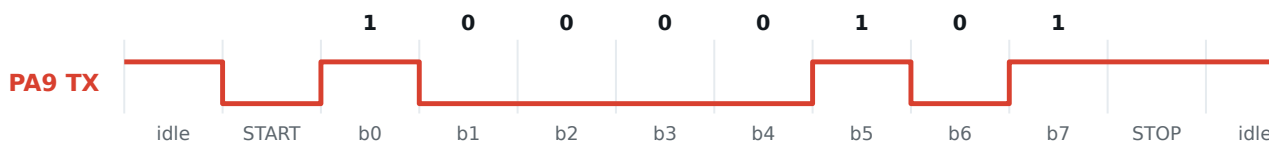
GPIOA->MODER	(1<<10) , (2<<18)	PA5 = output (LD2), PA9 = alternate function
GPIOA->AFR[1]	(7<<4)	AF7 selects USART1 on PA9 (AFR[1] is AFRH, 4 bits per pin)
USART1->BRR	1667	9600 Bd from a 16 MHz PCLK2
USART1->CR1	(1<<3) (1<<13)	TE and UE on the master; RE and UE ((1<<2) (1<<13)) on the slave
USART1->SR	bit 7 TXE , bit 5 RXNE	transmit register empty / a byte has arrived
USART1->DR	read or write	the data register; reading it also clears RXNE

5.3 Minimum register sequence - memorise this order

MASTER	SLAVE
1. RCC->AHB1ENR : GPIOA, GPIOC	1. RCC->AHB1ENR : GPIOA
2. RCC->APB2ENR : USART1	2. RCC->APB2ENR : USART1
3. PC13 pull-up, PA5 output	3. PA5 output
4. PA9 → alternate function, AFRH = 7	4. PA10 → alternate function, AFRH = 7
5. BRR = 1667	5. BRR = 1667 (same value!)
6. CR1 = TE UE	6. CR1 = RE UE
7. loop: read PC13, time it, wait TXE, write DR	7. loop: wait RXNE, read DR, blink

6 Signal timing

USART1 asynchronous character - 8N1, 9600 Bd, data byte 0xA1



0xA1 = 1010 0001 is shifted out LSB first, so the line carries 1 0 0 0 0 1 0 1.
 One bit lasts $1/9600 \text{ s} = 104.2 \mu\text{s}$; the complete 10-bit character takes about 1.04 ms.

Figure 2 – One asynchronous character on PA9. There is no clock wire: the receiver finds the bit boundaries from the falling edge of the start bit and its own 16x-oversampled baud-rate generator.

7 MASTER source code `uart_master.c`

Type this over the whole contents of `Core/Src/main.c` in the master project. It is self-contained: no other file to add, no header to edit. The comment block at the top is for you – skip it if you are short of time.

`uart_master.c`

```
1/* EXP 1  UART (USART1 async, 9600 8N1)  --- MASTER  ---  NUCLEO-F446RE
2 * Wiring : PA9 (CN10-21) -> slave PA10 (CN10-33)
3 *          GND (CN10-20) -> slave GND (CN10-20)
4 * B1 = PC13 (on board)  LD2 = PA5 (on board) */
5
6#include "stm32f446xx.h"
7
8static void delay_ms(uint32_t ms)
9{
10    SysTick->LOAD = 16000 - 1;          /* 1 ms at 16 MHz HSI */
11    SysTick->VAL  = 0;
12    SysTick->CTRL = 5;                  /* core clock, enable */
13    while (ms-- > 0) while (!(SysTick->CTRL & (1 << 16))); /* COUNTFLAG */
14    SysTick->CTRL = 0;
15}
16
17int main(void)
18{
19    uint32_t t;
20
21    RCC->AHB1ENR |= (1 << 0) | (1 << 2); /* GPIOA, GPIOC */
22    RCC->APB2ENR |= (1 << 4);           /* USART1 */
23
24    GPIOC->PUPDR |= (1 << 26);          /* PC13 pull-up (B1) */
25    GPIOA->MODER |= (1 << 10);          /* PA5 output (LD2) */
26    GPIOA->MODER |= (2 << 18);          /* PA9 alternate */
27    GPIOA->AFR[1] |= (7 << 4);          /* PA9 AF7 = USART1_TX */
28
29    USART1->BRR = 1667;                 /* 16000000 / 9600 */
30    USART1->CR1 = (1 << 3) | (1 << 13); /* TE, UE */
31
32    while (1) {
33        if (GPIOC->IDR & (1 << 13)) continue; /* B1 released -> wait */
34
35        t = 0;                          /* measure press length */
36        while (!(GPIOC->IDR & (1 << 13))) { delay_ms(10); t += 10; }
37
38        while (!(USART1->SR & (1 << 7))); /* TXE */
39        USART1->DR = (t >= 1000) ? 'L' : 'S';
40
41        GPIOA->ODR |= (1 << 5);          /* LD2 = command sent */
42        delay_ms(100);
43        GPIOA->ODR &= ~(1 << 5);
44    }
45}
```

8 SLAVE source code `uart_slave.c`

Type this over the whole contents of `Core/Src/main.c` in the slave project. It is self-contained: no other file to add, no header to edit. The comment block at the top is for you – skip it if you are short of time.

`uart_slave.c`

```
1/* EXP 1  UART (USART1 async, 9600 8N1)  --- SLAVE  --- NUCLEO-F446RE
2 * Wiring : PA10 (CN10-33) <- master PA9 (CN10-21)
3 *          GND (CN10-20) <- master GND (CN10-20)
4 * 'S' -> 3 blinks 150 ms      'L' -> 3 blinks 1000 ms (2 s spacing)  */
5
6#include "stm32f446xx.h"
7
8static void delay_ms(uint32_t ms)
9{
10     SysTick->LOAD = 16000 - 1;
11     SysTick->VAL  = 0;
12     SysTick->CTRL = 5;
13     while (ms-- > 0) while (!(SysTick->CTRL & (1 << 16)));
14     SysTick->CTRL = 0;
15}
16
17static void blink(uint32_t d)                /* d = half period      */
18{
19     for (int i = 0; i < 3; i++) {
20         GPIOA->ODR |= (1 << 5); delay_ms(d);
21         GPIOA->ODR &= ~(1 << 5); delay_ms(d);
22     }
23}
24
25int main(void)
26{
27     RCC->AHB1ENR |= (1 << 0);                /* GPIOA          */
28     RCC->APB2ENR |= (1 << 4);                /* USART1         */
29
30     GPIOA->MODER |= (1 << 10);               /* PA5 output (LD2) */
31     GPIOA->MODER |= (2 << 20);               /* PA10 alternate  */
32     GPIOA->AFR[1] |= (7 << 8);               /* PA10 AF7 = USART1_RX */
33
34     USART1->BRR = 1667;                      /* must match master */
35     USART1->CR1 = (1 << 2) | (1 << 13);      /* RE, UE          */
36
37     while (1)
38         if (USART1->SR & (1 << 5))          /* RXNE           */
39             blink(USART1->DR == 'L' ? 1000 : 150);
40}
```

9 Building it in STM32CubeIDE without touching CubeMX

1. **File → New → STM32 Project.** Open the *Board Selector* tab, type `NUCLEO-F446RE`, select it and press *Next*.
2. Name the project, keep *Targeted Language* = **C** and *Targeted Binary Type* = **Executable**, and set **Targeted Project Type = Empty**. This is the step that matters: an *Empty* project gives you the startup file, the linker script, `system_stm32f4xx.c` and the CMSIS register headers, but **no HAL, no LL drivers and no .ioc file**. There is nothing to click in CubeMX because there is no CubeMX.
3. If a dialog offers to initialise all peripherals, answer **No** – with an *Empty* project it normally does not appear at all.
4. Open `Core/Src/main.c`, select everything, delete it, and type the listing from this manual. `#include "stm32f446xx.h"` resolves through `Drivers/CMSIS/Device/ST/STM32F4xx/Include`, which the wizard already placed on the include path. There is no second file to create.
5. Build with **Ctrl + B**, connect the board, then *Run → Run As → STM32 C/C++ Application*.
6. Repeat for the second project with the other listing and flash the second board. Mark the boards **MASTER** and **SLAVE** – once they are off the PC they look identical.

If the build complains

`undefined reference to SystemInit` means the project was created without `system_stm32f4xx.c`; add `void SystemInit(void) { }` at the top of `main.c` – but only in that case, otherwise you get a duplicate symbol. `unknown type name 'uint32_t'` means the device header was not found: check that `STM32F446xx` is listed under *Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor*.

10 Running the pair with no PC attached

Nothing in these programs needs the debugger. As soon as the board is powered, the reset vector runs `main()` from flash, so the pair works on its own.

Option A – one USB supply per board (recommended)

1. Unplug both boards from the PC after flashing.
2. Plug each board's CN1 Mini-B socket into a USB power bank, a 5 V phone charger or a powered hub. Leave jumper **JP5 on U5V**, its factory position.
3. LD3 (red) lights on each board. Keep the protocol wires and the common ground exactly as they were during programming.

Option B – one 5 V source for both boards

1. Move **JP5 to E5V** on *both* boards.
2. Feed +5 V into **E5V** (CN7 pin 6) and 0 V into **GND** (CN7 pin 8 or CN10 pin 20) on both boards from the same supply.
3. The grounds are then already common, so the separate GND jumper becomes redundant – leaving it in place does no harm.

Safety

Never drive 5 V into the **+5V** pin while JP5 is on U5V *and* a USB cable is plugged in – that connects two supplies together. Pick one scheme and stay with it. BOOT0 must stay low (its default) so the MCU boots from flash.

11 Test procedure and observations

1. Flash the master listing to board 1 and the slave listing to board 2.
2. Power both boards down, then wire them as in the connection table. Check the ground wire twice – a missing ground is the single most common cause of a dead link.
3. Power both boards. LD3 (red) must be lit on each.
4. **Test 1 - short click.** Press and release B1 on the master in well under a second. The master's LD2 gives one short flash and the slave answers with three fast blinks.
5. **Test 2 - long press.** Hold B1 for at least one second, then release. The slave now blinks three times with one second on and one second off – two seconds from the start of one blink to the start of the next.
6. Time the slow pattern with a stopwatch. Repeat each test three times and record it.
7. **Test 3 - boundary.** Try a press of about one second and confirm which pattern you get. The threshold is the `t >= 1000` comparison in the master.
8. If a scope or logic analyser is available, capture the line(s) during one press and compare with the timing figure in section 6.

Trial	Press duration (ms)	Byte expected	Master LD2 flashed	Blinks counted	Measured spacing (s)	Pass
1						
2						
3						
4						
5						
6						

12 Troubleshooting

Symptom	Cause and remedy
The slave blinks but always with the fast pattern	The received byte is not 'L'. Either the baud rates differ, or the press really was shorter than 1000 ms.
Characters arrive as garbage	BRR differs between the boards, or one project still configures a PLL. Both must be 1667 at 16 MHz.
The slave never reacts	Ground is not common. Join GND (CN10-20) of the two boards – this is by far the most frequent fault.
The master's LD2 does not flash either	The master board is not running, or B1 was not detected. Check LD3 (red) is lit and that you typed <code>GPIOC->PUPDR = (1 << 26);</code> – without the pull-up, PC13 floats and never reads as pressed.
Blink timing is wrong (much faster or slower than 2 s)	SYSClk is not 16 MHz. Make sure no <code>SystemClock_Config()</code> or PLL setup was left in <code>main.c</code> , and that <code>SysTick->LOAD</code> is 16000 – 1.
Both boards blink but the roles are swapped	The master listing was flashed to the slave board. Label the boards.
A long press is read as a short one	You released before 1000 ms. The threshold is the literal 1000 in the master's <code>t >= 1000</code> test – change it there if your lab asks for a different value.

13 Review questions

1. Why does an asynchronous link need a start bit at all?

2. Compute BRR for 115200 Bd at PCLK2 = 16 MHz and give the resulting error.
3. What is the difference between the TXE and TC flags?
4. Why are TX and RX crossed here but SCK is not crossed in SPI?
5. How many bits actually travel on the wire for one 8N1 character?
6. What happens if the two boards share data wires but not ground?
7. Which register bit tells you a byte has been received, and how is it cleared?
8. How would you make this receive path interrupt-driven instead of polled?